

Application, Limitations, and Future

The Azubi Method, Part 3

General Applicability, LLM Integration, and Open Questions

General Applicability

The Azubi Method is not limited to a single domain. It can be applied to any result produced by an expert or an expert system. The reason is simple: silent assumptions are not a domain-specific problem. They are a knowledge problem. Wherever expertise is applied, blind spots emerge. And wherever blind spots emerge, context-free questions can make them visible.

Assessments and analyses. An AI agent produces a risk assessment. The Azubi asks: “Three risks were rated as ‘low’—by what standard?” The question forces transparency about the evaluation criteria. Often it turns out that the standard was never defined—it was a silent assumption of the expert.

Audit reports. An audit system evaluates a result against a catalog. The Azubi asks: “14 audit items were skipped—is that stated in the report?” The question reveals what the report does not say. In practice, what an audit did not examine is often more important than what it did.

Security analyses. A security agent reviews a system and reports no anomalies. The Azubi asks: “Why was area X not reviewed?” The question makes visible what lay outside the defined review scope. The boundary of the review scope is itself an assumption that is rarely questioned.

Detection of malicious patterns. An LLM analyzes data for anomalies and finds nothing suspicious. The Azubi asks: “Why is this pattern treated as normal? What if it isn’t?” The question challenges the definition of “normal”—and that is precisely where malicious patterns can hide, remaining undetected because they fall within expected parameters. An expert who has seen hundreds of normal cases develops a notion of what “normal” looks like. Everything that fits this notion goes unexamined. This is the attention threshold in the security domain.

Strategic decisions. An AI agent produces a recommendation. The Azubi asks: “The recommendation is based on three assumptions—what happens if the second one is wrong?” The question makes silent dependencies visible.

Summaries and consolidations. An agent summarizes results from multiple sources. The Azubi asks: “Source A says X and Source B says Y—why does the

summary only contain X?” The question reveals whether the consolidating agent smoothed over a contradiction instead of making it visible.

The framework itself. Anyone who has created a ruleset can ask the Azubi whether the ruleset makes sense. The method can question itself, and that is precisely what makes it robust. A system that also exposes its own foundations to the naive gaze has fewer blind spots than one that only reviews its results.

The pattern is always the same: the task description and the result are the only inputs. The Azubi knows nothing about the path between them. And that is precisely what enables it to ask the questions that everyone who knows the path no longer asks.

Related Approaches

The Azubi Method is related to several existing approaches but differs in a specific point:

The Socratic Method asks targeted, informed questions to make the conversation partner think. The Azubi Method asks naive questions without context. The questioner does not know what the “correct” answer is. Rubber duck debugging forces self-reflection through explanation. The Azubi Method poses questions from the outside, without requiring the reviewer to explain their own process. The fresh-eyes review—a new reviewer looks at the result—is the human equivalent. The Azubi Method formalizes and systematizes this principle and makes it repeatable and wear-free with AI agents. Socratic Debugging (Al-Hossami et al., 2023) uses Socratic questioning to guide programming beginners toward finding their own bugs. It aims at learning, not quality assurance.

Based on my research, there is no formalized approach that employs structured naïveté without context as a systematic review mechanism. The Azubi Method is, in this specific respect—context-free questioning as an architectural principle—an independent contribution.

Limitations—Honestly Stated

The Azubi Method finds silent assumptions. It does not find problems that only become visible under real-world conditions. No usability problems with real users. No vulnerabilities that only appear through active stress testing.

The observed 100% hit rate—not a single Azubi question was superfluous—is based on over 30 questions in a specific context. Whether the same rate is achievable in other domains is an open question that remains honestly marked as unanswered. The method finds what it finds. It does not guarantee that there is nothing it cannot find. No tool can.

The method is a tool, not a solution. It covers a specific error class that other methods do not cover. It does not make other methods obsolete. And it does not make humans obsolete—in the end, the human decides what to do with the questions.

And: everything depends on the specific language model. How well an Azubi agent asks questions, how deeply an expert agent reviews, how susceptible a model is to sycophancy or hallucinations—this varies between models and even between versions of the same model. The methods in this paper address problems that are real and measurable today. Whether they will apply to the same degree in two years is an open question.

For LLM Developers

The Azubi Method describes a mechanism that must be applied manually today. The developer creates their context-free instance, opens a new chat, copies task description and result, asks for questions. This works. But it does not scale. A hundred teams independently rebuilding the same mechanism are a hundred individual solutions that all work to varying degrees.

The Azubi Method is the most obvious candidate for direct integration into language models: a model that automatically performs a context-free self-check after every expert output. An internal instance that sees only the result and poses a counter-question. This would eliminate the manual step and make the method accessible to every user without them needing to know about it.

The vision would be: a model that automatically poses a context-free counter-question after a review would no longer need an external Azubi. A model that deliberately shifts its perspective during repeated reviews would have rotation as a default. If that happens, this paper would no longer be an instruction manual but a description of what models do anyway. That would be the best possible outcome.

Whether this is technically feasible and whether it can be meaningfully implemented in model architecture—I cannot judge. I am a developer, not an AI researcher. But the observation that the described problems are real and measurable today is at least a starting point for those working on the architecture.

Beyond that, there are three concrete areas where developers of language models can make progress:

Context understanding over long inputs. Models do not improve with growing context—they degrade. A phenomenon described in research as context rot. Better context understanding across the full window length would improve the effectiveness of every review method.

Resistance to context poisoning. When a false assumption enters the context and amplifies itself through subsequent responses, it poisons the entire system. Models that recognize when a context entry contradicts their own knowledge, and name that contradiction rather than confirm it, would be a fundamental advance.

Reduced sycophancy in multi-agent scenarios. The confirmation tendency between agents is a potentially larger problem than that between human and AI. Developing models that become more critical rather than more agreeable in agent-to-agent communication would be a lever that extends far beyond this paper.

The Philosophy Behind It

The foundation is 2,500 years old. Socrates claimed to know nothing. And therefore asked the questions that forced his conversation partners to examine their own assumptions.

The Azubi Method is the systematization of this principle for the age of AI agents: knowledge creates blind spots. Ignorance creates questions. And questions find more than answers.

Conclusion

Every review method makes the mesh tighter, not gap-free. Automated checks find known patterns. Experts find expected logic errors. The Azubi Method finds silent assumptions—the errors that produce no error message, no warning, no visible symptom. The errors that live unnoticed in the system for weeks or months.

The method is applicable today. Without infrastructure, without budget, without specialized knowledge. A new chat, a task description, a result, and a request for

questions. Two minutes. One API call. And the honest willingness to engage with the answers.

There is no 100%. There are only tighter meshes. And every mesh that catches an error that would have slipped through before has been worth it.

The right questions are more valuable than the right answers.

References

Foundational Concepts

ELIZA Effect: Weizenbaum, J. (1966). "ELIZA—A Computer Program for the Study of Natural Language Communication Between Man and Machine." *Communications of the ACM*, 9(1), 36–45.

Groupthink: Janis, I. L. (1972). "Victims of Groupthink: A Psychological Study of Foreign-Policy Decisions and Fiascos." Houghton Mifflin.

Socratic Method: Plato, c. 399 BCE. Transmitted in the Dialogues, particularly the *Apology of Socrates*.

AI Code Quality

CodeRabbit (2025). "State of AI vs Human Code Generation Report." 470 real pull requests.

Faros AI (2025). "The AI Productivity Paradox Research Report." 10,000+ developers.

METR (2025). Randomized study with experienced open-source developers. 19% slower, subjectively perceived as 20% faster.

Sycophancy and Reward Hacking

Sharma, M. et al. (2023). "Towards Understanding Sycophancy in Language Models." *ICLR 2024*.

Lechmazur Sycophancy Benchmark (2026). Contrarian Contradiction.

Beigi, M. et al. (2025). "SMART: Sycophancy Mitigation Through Reinforcement Learning."

Denison, C. et al. (2024, Anthropic). "Sycophancy to Subterfuge: Investigating Reward-Tampering in Large Language Models."

Context Problems in LLMs

Chroma Research (2025). "Context Rot: How Increasing Input Tokens Impacts LLM Performance."

Breunig, D. (2025). "How Long Contexts Fail."

Related Approaches

Al-Hossami, E. et al. (2023). "Socratic Questioning of Novice Debuggers." BEA Workshop, ACL 2023.

About the Author

Andreas Schmidt conducts independent research on AI applications and works part-time in software development. Through Andreas Schmidt Arts, he combines marketing, design, and coding. The method described in this paper emerged from practical experience with AI agent systems in software quality assurance.

Contact: info@andreas-schmidt-arts.de

Transparency Note

A language model assisted in the creation of this paper. The concepts, observations, and methods originate from the author's practical work. The structuring, formulation, and source research were conducted in collaboration with an LLM. All content was reviewed, directed, and approved by the author.